Redes de Comunicação

Meta Final



Discentes:

Cláudio Manuel Gomes Catarino 2022224320

Miguel Alexandre Pontes Calado 2022228235

Docente:

Vítor Manuel Garcia do Nascimento Graveto

Introdução

Nesta meta final do projeto, era-nos pedido que implementássemos as funcionalidades todas, tanto de rede como de programação necessárias para que o sistema pretendido de gerenciamento de turmas funcionasse.

Desenvolvimento do Trabalho

A primeira parte de desenvolvimento deste projeto baseou-se na criação de um cliente tcp, de modo a permitir a comunicação entre alunos ou professores com o servidor. Como não nos era pedido para desenvolver o cliente udp, optámos por não o fazer, usando o netcat para estabelecer este tipo de ligação com o server.

Após a finalização do código do cliente, passámos para o desenvolvimento daquele que seria o código do servidor. Para isto, decidimos usar dois ficheiros .c, um (server.c) com a inicialização do servidor e o processamento do tipo de comunicação que foi estabelecida, usando para isso dois processos, um responsável pela comunicação tcp e outro pela udp. O outro ficheiro .c (COMANDOS.c) tem o processamento dos comandos que o utilizador inserir. Para estabelecer a conexão entre estes ficheiros foi usado um ficheiro .h (COMANDOS.h), que contém os cabeçalhos das funções que o servidor necessita e que estão a ser processadas no segundo ficheiro mencionado.

Os comandos aceites por administradores, que usam udp como protocolo de comunicação, são:

- ADD_USER: De forma a implementar este comando verificamos os parâmetros inseridos pelo admin (username, password e usertype), usando a função escreveUserFicheiro para guardar os dados do novo user no ficheiro logins.txt

- DEL: Para este comando criámos a função removeUtilizador que confirma se o username e a password existem no ficheiro, tendo estes de estar diretamente associados, e, caso se confirme a existência de um user com o username e a password, ele é removido do ficheiro logins.txt.

- LIST: Foi implementada a função mostraUtilizadores que percorre o ficheiro logins.txt e imprime todos os users lá presentes (username e password).

-QUIT_SERVER: Responsável por fechar o servidor.

Se o user que se logar for um aluno, a comunicação é feita via tcp e os comandos permitidos são:

- LIST_CLASSES: Foi implementada a função mostraClasses, que percorre o ficheiro classes.txt e imprime todas as classes existentes (nome e tamanho total).

- SUBSCRIBE_CLASS: A função subscreverTurma é responsável pelo funcionamento deste comando. Nesta função, é criado um ficheiro temporário que vai ser usado para transferir a informação do ficheiro classes.txt para ele, de modo a permite acrescentar o novo aluno à turma pretendida. Após a criação deste ficheiro, é verificado se o aluno já está inscrito na turma pretendida e se a turma não está preenchida e caso nenhum deles aconteça, é adicionado o novo aluno à turma, sendo depois o ficheiro temporário renomeado para classes.txt e o antigo ficheiro apagado.

- LIST_SUBSCRIBED: Faz uso da função imprimirClassesAluno, na qual o ficheiro classes.txt é percorrido e é impresso as turmas onde o aluno está inscrito (nome e endereço multicast da turma).

Para o caso de ser um professor a fazer o login, é usado tcp como o protocolo de comunicação, e apenas os seguintes comandos são permitidos:

- CREATE_CLASS: É usada a função escreveClassFicheiro que abre o ficheiro classes.txt e, usando a função verificaClasses vê se a classe que se pretende criar já existe. Caso esta não exista, acrescenta a classe pretendida ao ficheiro (endereço multicast, nome e tamanho). Para atribuir um endereço multicast é usada a função criaMulticast que criar um endereço e verifica se este já não está atribuído a uma outra classe. Se estiver cria outro, verificando novamente e atribui o primeiro endereço que estiver disponível.

- LIST_CLASSES: Similarmente ao que acontece com este comando no caso do aluno, é usada a função mostraClasses cujo funcionamento já foi previamente explicado.

- LIST_SUBSCRIBED: A funcionalidade deste comando é idêntico ao do aluno, mudando apenas a forma como o professor é identificado no ficheiro, sendo este antecedido por “;:”, ao contrário do aluno que é apenas antecedido por “;”.

- SUBSCRIBE_CLASS: É também idêntico ao que acontece com o aluno, contudo utiliza a função subscrever TurmaProfessor, na qual é confirmar, através da função professor_existe verifica se aquele professor já está atribuído à turma pretendida, e caso não esteja atribui e escreve no fim da linha o nome do professor.

- SEND: Através da função enviarMulticastUDP, e caso a turma exista e se o professor está atribuído àquela turma, envia uma mensagem para todos os alunos da turma, utilizando o endereço multicast para realizar a operação pretendida.

Decisões adotadas

Quando criamos os endereços multicast, a primeira parte destes é 239, uma vez que valores antes deste podem resultar em problemas.

Em algumas funções foram criados ficheiros temporários, sendo estes depois renomeados e o ficheiro antigo apagado. Tomámos esta decisão de modo a facilitar a adição de novas informações aos ficheiros de texto.

Para melhor identificação de quais nomes correspondem a professores ou alunos, nos ficheiros, decidimos adicionar como separador “;:” caso seja professor e “;” caso seja aluno. Por exemplo, “;:joana;jose”, indicaria que joana era a porfessora e jose um aluno.

Possíveis Problemas

Um dos possíveis problemas que se pode encontrar é um delay no aparecimento da mensagem output após a escrita de um comando. Este erro não afeta a funcionalidade do programa, uma vez que o comando é processado e caso seja inserida qualquer caracter ou frase na consola, a mensagem aparece e, após isto se tentarmos confirmar que a operação aconteceu é possível verificar o sucesso da mesma. Isto pode dever-se a um mau gerenciamento dos buffers pela nossa parte ou por bugs da própria máquina, uma vez que estes erros nem sempre acontecem e são mais frequentes quando o GNS3 é usado para correr os códigos.